



LeetCode 2381: Shifting Letters II

1. Problem Title & Link

Title: LeetCode 2381: Shifting Letters II

Link: <https://leetcode.com/problems/shifting-letters-ii/>

2. Problem Statement (Short Summary)

You are given:

- A string `s`
- An array `shifts`

Each shift contains:

`[start, end, direction]`

- `direction = 1` → shift characters forward
- `direction = 0` → shift characters backward

For every operation, all characters from `start` to `end` must be shifted.

Return the final string after performing all shifts.

3. Examples (Input → Output)

Example 1

Input:

`s = "abc"`

`shifts = [[0,1,0],[1,2,1],[0,2,1]]`

Output:

`"ace"`

Explanation:

Operation 1:

`abc`

↓

`zac`

Operation 2:

`zac`

↓

`zbd`

Operation 3:

`zbd`

↓

`ace`

**Example 2**

Input

`s = "dztz"`

`shifts = [[0,0,0],[1,1,1]]`

Output

`"catz"`

Example 3

Input

`s = "aaa"`

`shifts = [[0,2,1]]`

Output

`"bbb"`

4. Constraints

- $1 \leq s.length \leq 5 \times 10^4$
- $1 \leq shifts.length \leq 5 \times 10^4$
- `s` contains lowercase English letters
- $O(n \times m)$ will TLE
- Need near $O(n)$

5. Core Concept (Pattern / Topic)**Prefix Sum (Difference Array)**

Related Concepts:

- Range Update
- Sweep Line
- Prefix Accumulation

6. Thought Process (Step-by-Step Explanation)**Brute Force**

For every shift:

for each operation:

update every character in range

Complexity:



$O(n \times \text{shifts})$

Worst Case:

50000×50000

Too slow.

Optimized Thinking

Instead of updating every character immediately:

Store only:

range starts

range ends

using a Difference Array.

For a forward shift:

$\text{diff}[\text{start}] += 1$

$\text{diff}[\text{end}+1] -= 1$

For a backward shift:

$\text{diff}[\text{start}] -= 1$

$\text{diff}[\text{end}+1] += 1$

Then compute prefix sums.

Each position now knows total shifts applied.

Why It Works

Prefix sum converts:

range updates

into

single pass accumulation

allowing all operations to be processed efficiently.

7. Visual / Intuition Diagram (ASCII Diagram)

```
String:
a b c d e
Shift:
[1,3,+1]
Difference Array:
0 1 0 0 -1
Prefix Sum:
0 1 1 1 0
Actual effect:
a c d e e
Only positions 1..3 shifted.
```



8. Pseudocode (Language Independent)

```
Create diff array

For every shift:
    if forward:
        diff[start] += 1
        diff[end+1] -= 1
    else:
        diff[start] -= 1
        diff[end+1] += 1

Compute prefix sum

For each character:
    apply shift
    handle wraparound

Return string
```

9. Code Implementation

✓ Python

```
class Solution:
    def shiftingLetters(self, s: str, shifts):
        n = len(s)

        diff = [0] * (n + 1)

        for start, end, direction in shifts:

            if direction == 1:
                diff[start] += 1
                diff[end + 1] -= 1
            else:
                diff[start] -= 1
                diff[end + 1] += 1

        current = 0
        result = []

        for i in range(n):
            current += diff[i]

            new_char = (
                (ord(s[i]) - ord('a') + current) % 26
            ) + ord('a')

            result.append(chr(new_char))
```



```
return "".join(result)
```

✓ Java

```
class Solution {
    public String shiftingLetters(String s, int[][] shifts) {

        int n = s.length();

        int[] diff = new int[n + 1];

        for (int[] shift : shifts) {

            int start = shift[0];
            int end = shift[1];
            int direction = shift[2];

            if (direction == 1) {
                diff[start]++;
                diff[end + 1]--;
            } else {
                diff[start]--;
                diff[end + 1]++;
            }
        }

        StringBuilder result = new StringBuilder();

        int current = 0;

        for (int i = 0; i < n; i++) {

            current += diff[i];

            int value =
                ((s.charAt(i) - 'a') + current) % 26;

            if (value < 0)
                value += 26;

            result.append((char) ('a' + value));
        }

        return result.toString();
    }
}
```



10. Time & Space Complexity

Time Complexity

$O(n + m)$

where

- n = length of string
- m = number of shifts

Space Complexity

$O(n)$

Difference array.

11. Common Mistakes / Edge Cases

Mistake 1

Forgetting:

$\text{end} + 1$

in difference array.

Mistake 2

Negative modulo issue.

Java:

$-1 \% 26 = -1$

Need:

$\text{if}(\text{value} < 0)$

$\text{value} += 26;$

Mistake 3

Using brute force.

Will exceed time limit.

12. Detailed Dry Run (Step-by-Step Table)

Input:

$s = \text{"abc"}$

$\text{shifts} = [[0,1,0],[1,2,1],[0,2,1]]$

Difference Array Updates:

Operation	Diff
$[0,1,0]$	$-1 \ +1$



[1,2,1]	+1 -1
[0,2,1]	+1 -1

Final diff:

[0,1,0,-1]

Prefix:

Index	Prefix
0	0
1	1
2	1

Apply:

Char	Shift	Result
a	0	a
b	1	c
c	1	e

Output:

"ace"

13. Common Use Cases (Real-Life / Interview)

Range Salary Updates

Increase salary of employees

from index L to R

Game Buff Effects

Apply effect to players

within range

Traffic Updates

Update vehicles on road segments

14. Common Traps (Important!)

- Using direct range modification
- Missing modulo 26
- Forgetting negative shifts
- Not allocating n+1 diff array

15. Builds To (Related LeetCode Problems)



1. LC 303 — Range Sum Query
2. LC 560 — Subarray Sum Equals K
3. LC 1094 — Car Pooling
4. LC 1109 — Corporate Flight Bookings
5. LC 2381 — Shifting Letters II

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	$O(n \times m)$	$O(1)$	TLE
Range Update Simulation	$O(n \times m)$	$O(1)$	Slow
Difference Array + Prefix Sum	$O(n + m)$	$O(n)$	Optimal

17. Why This Solution Works (Short Intuition)

Difference array efficiently stores all range updates. After computing prefix sums, each position knows exactly how many shifts should be applied, allowing the final string to be built in one pass.

18. Variations / Follow-Up Questions

Follow-Up 1

What if uppercase letters are included?

Need separate handling for:

A-Z

a-z

Follow-Up 2

What if updates arrive continuously?

Use:

Segment Tree

Fenwick Tree

Follow-Up 3

What if queries ask for intermediate strings?

Use lazy propagation segment tree.